Introduction

This notebook presents the basic data cleaning process for the PlaySafe project. The aim of this step is to prepare the collected LaLiga player dataset for the next stages of the project.

The raw dataset contains player characteristics, playing time, season information, and injury-related information. The dataset has been selected from a GitHub repository which contains raw data about soccer where I selected this file; https://github.com/Michael-starz/football-injury-scraper/blob/main/laliga_players.csv?

Cleaning objectives

The main objectives of this notebook are to:

- create a cleaned dataset;
- identify basic data issues;
- standardise missing values;
- handle clearly inconsistent values;
- convert relevant variables to appropriate formats;
- remove the redundant variables;
- verify the final structure and remaining missing values;
- prepare the dataset for the next stages of the project.

This cleaning process is intentionally kept basic. Issues that require further investigation, interpretation, or feature engineering will be addressed in later stages rather than during this initial cleaning process.

In [1]:
```
import pandas as pd
import numpy as np
import csv

# Load the raw data from the Excel file
raw = pd.read_excel("../data/raw/laliga_final_dataset.csv.xlsx", header=None, dt


rows = list(csv.reader(raw[0].tolist())) # splits each string into it fields
```

The initial attempt to split the dataset using commas resulted in 16 columns instead of the expected 15 columns.

This occurred because some date values contain commas, for example "Nov 23, 2024". A simple comma-based split therefore incorrectly separated the "to_date" variable into two columns.

To avoid losing information, the original dataset is reloaded and the columns will be reconstructed using a controlled approach.

In [2]:
```
df_raw = pd.read_excel("../data/raw/laliga_final_dataset.csv.xlsx", header=None)

print("Shape:", df_raw.shape)
df_raw.head(3).T
```

```
Shape: (3714, 1)
```

Out[2]:

	0	1	2
0	player_name,date_of_birth/age,height,position,...	Thibaut Courtois,May 11 1992 (32),200 m,Goalke...	Thibaut Courtois,May 11 1992 (32),200 m,Goalke...

The raw file contains 3,714 rows and one column. The first row contains the names of the 15 original variables, while each subsequent row contains a complete record stored as a comma-separated string.

A simple comma-based split cannot be used because some values, especially dates such as "Nov 23, 2024", also contain commas.

Therefore, the rows will be parsed using a CSV parser (Comma separated values) , which correctly handles commas inside quoted values.

In [3]:
```
import csv

rows = [] # create an empty list to store the rows

for row in df_raw[0]:
    rows.append(next(csv.reader([row])))

print("Number of rows:", len(rows))
print("Columns in first row:", len(rows[0]))
```

```
Number of rows: 3714
Columns in first row: 15
```

Before creating the final DataFrame, we check that every row contains exactly 15 values.

In [4]:
```
row_lengths = [len(row) for row in rows]

print("Minimum number of columns:", min(row_lengths))
print("Maximum number of columns:", max(row_lengths))
print("Rows with 15 columns:", sum(length == 15 for length in row_lengths))
print("Rows with a different number of columns:", sum(length != 15 for length in
```

```
Minimum number of columns: 15
Maximum number of columns: 15
Rows with 15 columns: 3714
Rows with a different number of columns: 0
```

Now we finally parsed the 15 columns we wanted to get we can assign them their corresponding original name and create a pandas DatasetFrame

In [5]:
```
columns = [
    "player_name",
    "date_of_birth_age",
```

```
    "height",
    "position",
    "current_age_x",
    "minutes_played",
    "season",
    "injury",
    "from_date",
    "to_date",
    "days_out",
    "matches_missed",
    "injury_year",
    "current_age_y",
    "age_at_injury"
]

df = pd.DataFrame(rows[1:], columns=columns)

print("Shape:", df.shape)
df.head()
```

```
Shape: (3713, 15)
```

Out[5]:

	player_name	date_of_birth_age	height	position	current_age_x	minutes_played	s
0	Thibaut Courtois	May 11 1992 (32)	200 m	Goalkeeper	32	2970.0	
1	Thibaut Courtois	May 11 1992 (32)	200 m	Goalkeeper	32	2970.0	
2	Thibaut Courtois	May 11 1992 (32)	200 m	Goalkeeper	32	513.0	
3	Thibaut Courtois	May 11 1992 (32)	200 m	Goalkeeper	32	513.0	
4	Thibaut Courtois	May 11 1992 (32)	200 m	Goalkeeper	32	4920.0	

In [6]:

```
# Check the data types of all columns
df.dtypes
```

Out[6]:

```
player_name          str
date_of_birth_age    str
height               str
position             str
current_age_x        str
minutes_played       str
season               str
injury               str
from_date            str
to_date              str
days_out             str
matches_missed       str
injury_year          str
current_age_y        str
age_at_injury        str
dtype: object
```

Here wwe can see how all columns are stored as strings

In [7]:
```
# Count empty strings in each column
(df == "").sum()
```

Out[7]:
```
player_name            0
date_of_birth_age      0
height                17
position               0
current_age_x          0
minutes_played         9
season                 9
injury              1937
from_date           1937
to_date             1939
days_out            1937
matches_missed      1937
injury_year         1937
current_age_y       1937
age_at_injury       1937
dtype: int64
```

The dataset was checked for empty strings because missing information in the raw data is represented by empty values.

In [8]:
```
df["height"].unique()[:20]
# look at some height values to identify the format
```

Out[8]:
```
<StringArray>
['200 m', '191 m', '186 m', '190 m', '180 m', '184 m', '169 m', '173 m',
 '188 m', '182 m', '172 m', '179 m', '176 m', '174 m', '175 m', '170 m',
 '178 m', '187 m', '195 m', '192 m']
Length: 20, dtype: str
```

The values in `height` were inspected to identify inconsistent formats. The values are stored as strings and include the unit "m", for example `"200 m"`, `"191 m"` and `"187 m"`.

In [9]:
```
#check if non-empty height values end with " m"
height_format_check = df.loc[df["height"] != "", "height"].str.endswith(" m")
print("Total rows with height:", df["height"].notna().sum())
print("Values ending with ' m':", height_format_check.sum())
print("Values not ending with ' m':", (~height_format_check).sum())
```

```
Total rows with height: 3713
Values ending with ' m': 3696
Values not ending with ' m': 0
```

The format of the height variable was checked to determine whether the non-empty values consistently include the unit "m".

The results show that 3,696 non-empty values follow the expected "xxx m" format, while 17 records contain empty values. No non-empty values were found with an inconsistent format.

The unit "m" can therefore be removed during the cleaning process and the remaining values converted to a numerical data type.

In [10]:
```
df["days_out"].unique()[:20]
```

Out[10]:
```
<StringArray>
[ '32 days',  '19 days',  '41 days', '235 days',   '5 days',  '12 days',
  '11 days',   '1 days',  '29 days',         '',   '9 days',  '13 days',
 '266 days',   '8 days',  '18 days', '214 days',   '7 days',  '27 days',
  '16 days',  '31 days']
Length: 20, dtype: str
```

The `days_out` variable was inspected to determine how injury duration is represented. We identified that the values contain the number of days followed by the text `"days"`.

The text needs to be removed before the variable can be converted to a numerical format.

In [11]:
```
df["season"].unique()
```

Out[11]:
```
<StringArray>
['24/25', '23/24', '22/23', '21/22', '20/21', '18/19', '17/18', '16/17',
 '15/16', '14/15', '13/14', '12/13', '19/20',      '']
Length: 14, dtype: str
```

The `season` variable contains season identifiers such as `"24/25"`, `"23/24"` and `"22/23"`. Nine records contain an empty value.

In [12]:
```
# Count "-" values in matches_missed
(df["matches_missed"] == "-").sum()
```

Out[12]:
```
np.int64(96)
```

In [13]:
```
(df["matches_missed"] == "?").sum()
#check for "?" values in matches_missed
```

Out[13]:
```
np.int64(4)
```

The `matches_missed` variable was checked for the empty values, the inspection found 96 occurrences of `"-"` and 4 of `"?"`

These values cannot be converted directly to numbers and therefore need to be treated as missing values

In [14]:
```
df["matches_missed"].unique()
```

Out[14]:
```
<StringArray>
[ '3',  '2',  '6', '42',  '1',  '5',  '4',  '-',   '', '46', '45', '24', '58',
  '7', '18', '11',  '8', '10', '12', '50', '15', '14',  '9', '35', '13', '48',
 '19', '16', '28', '23', '64', '22', '33',  '?', '40', '26', '21', '32', '27',
 '31', '25', '17', '44', '29', '34', '41', '20', '30', '37', '49', '51', '39',
 '60', '36', '47', '38', '43']
Length: 57, dtype: str
```

The unique values in `matches_missed` were inspected to identify all different formats. Most values are numerical strings, but the variable also contains `"-"`, `"?"` and empty values.

Therefore, the column cannot be converted directly to a numerical data type without first handling these non-numerical values.

In [15]:
```
df[["current_age_x", "current_age_y"]].dtypes

# check the data types of the current_age_x and current_age_y columns
```

Out[15]:
```
current_age_x    str
current_age_y    str
dtype: object
```

In [16]:
```
df[["current_age_x", "current_age_y"]].isna().sum()
# check for missing values in the current_age_x and current_age_y columns
```

Out[16]:
```
current_age_x    0
current_age_y    0
dtype: int64
```

The data types of current_age_x and current_age_y were compared. Both variables are initially stored as strings.

The two variables appear to represent the same type of information, but they use different representations in the raw dataset, such as "32" and "32.0". At this stage, empty strings are not yet recognised as missing values, which is why both variables show zero missing values in the check above.

After standardising empty strings as NaN, current_age_y contains many missing values (1937), while current_age_x does not (0). Therefore, current_age_y can be removed as the redundant age variable.

In [17]:
```
df["date_of_birth_age"].head(10)
```

Out[17]:
```
0    May 11 1992 (32)
1    May 11 1992 (32)
2    May 11 1992 (32)
3    May 11 1992 (32)
4    May 11 1992 (32)
5    May 11 1992 (32)
6    May 11 1992 (32)
7    May 11 1992 (32)
8    May 11 1992 (32)
9    May 11 1992 (32)
Name: date_of_birth_age, dtype: str
```

The `date_of_birth_age` variable was inspected because it contains both date-of-birth and age information in the same field. For example, values are formatted as `"May 11 1992 (32)"`.

This combined format is not suitable for direct numerical analysis. However, separating this variable into individual features would be considered feature preparation rather than

basic cleaning.

Initial Inspection Summary

The initial inspection identified several data-quality issues that need to be addressed before continuing with the analysis The dataset contains 3,713 rows and 15 columns. Initially, all columns were stored as strings, including numerical variables and dates.

The first inspection focused on identifying:

- missing values;
- inconsistent data formats;
- duplicated or potentially redundant variables;
- variables requiring type conversion;
- possible data quality and encoding issues.

Main findings

- height: contains values such as "187 m" and 17 empty values. The numeric part appears to represent the player's height.
- date_of_birth_age: combines the date of birth and age in a single field.
- minutes_played: contains numerical values stored as strings and has 9 empty values.
- season: contains values such as "24/25" and has 9 empty values.
- injury: contains 1,937 empty values.
- from_date and to_date: contain dates stored as strings and have missing values.
- days_out: contains values such as "32 days" and 1,937 empty values.
- matches_missed: contains numerical values stored as strings, as well as "-" and empty values. There are occurrences of `"-"` and `"?"`.
- injury_year: contains years stored as strings, with missing values.
- current_age_x and current_age_y: appear to represent the same concept, but they use different formats ("32" versus "32.0"). current_age_x contains no empty values, while `current_age_y contains missing values.
- age_at_injury: contains numerical values stored as strings and missing values.

These findings define the basic cleaning steps that will be performed next. The cleaning will focus on making the existing variables usable and consistent while avoiding unnecessary feature engineering at this stage.

Basic data cleaning

For this stage of the project, a basic data cleaning process is carried out to prepare the dataset for the next stages of the project. This cleaning will manage the following findings:

- Standardise empty strings → NaN
- Clean matches_missed (- and ? → missing, numeric conversion)

- Verify the important data types
- Check remaining missing values + final shape

Not all identified data issues will be corrected at this stage. The purpose of this cleaning process is to perform only the basic preparation necessary for Iteration Zero, such as standardising missing values and data types and handling clearly inconsistent format The other isseus will be adressed in later stages of the project.

In [18]:
```
# replace missing values with NaN
df = df.replace("", np.nan)

print(df.isna().sum())
```

```
player_name             0
date_of_birth_age       0
height                 17
position                0
current_age_x           0
minutes_played          9
season                  9
injury               1937
from_date            1937
to_date              1939
days_out             1937
matches_missed       1937
injury_year          1937
current_age_y        1937
age_at_injury        1937
dtype: int64
```

During the initial inspection, some missing values were represented as empty strings ("") rather than standard missing values (NaN). To make missing values consistent and easier to handle, all empty strings are converted to NaN.

In [19]:
```
df["matches_missed"] = pd.to_numeric(
    df["matches_missed"].replace(["-", "?"], np.nan),
    errors="coerce"
)
# convert matches_missed to numeric, replacing "-" and "?" with NaN
print(df["matches_missed"].dtype)
```

```
float64
```

The matches_missed variable contains numerical values as well as special values such as "-" and "?". Since these special values cannot be interpreted as numerical values, they are converted to missing values (NaN).

In [20]:
```
# convert columns to appropriate data types
df["height"] = pd.to_numeric(
    df["height"].astype("string").str.replace(" m", "", regex=False),
    errors="coerce"
)

df["minutes_played"] = pd.to_numeric(
    df["minutes_played"],
    errors="coerce"
```

```
)

df["current_age_x"] = pd.to_numeric(
    df["current_age_x"],
    errors="coerce"
)

df["days_out"] = pd.to_numeric(
    df["days_out"].astype("string").str.replace(" days", "", regex=False),
    errors="coerce"
)

df["injury_year"] = pd.to_numeric(
    df["injury_year"],
    errors="coerce"
)

df["age_at_injury"] = pd.to_numeric(
    df["age_at_injury"],
    errors="coerce"
)

df["from_date"] = pd.to_datetime(
    df["from_date"],
    errors="coerce"
)

df["to_date"] = pd.to_datetime(
    df["to_date"],
    errors="coerce"
)

print(df.dtypes)
```

```
player_name                 str
date_of_birth_age           str
height                    Int64
position                    str
current_age_x             int64
minutes_played          float64
season                      str
injury                      str
from_date           datetime64[us]
to_date             datetime64[us]
days_out                  Int64
matches_missed          float64
injury_year             float64
current_age_y               str
age_at_injury           float64
dtype: object
```

Several numerical and date variables are still stored as text. To make the dataset suitable for basic analysis, these variables are converted to appropriate numeric or datetime data types.

In [21]:
```
# Remove the redundant age column

df = df.drop(columns=["current_age_y"])
```

```
print("Shape after removing redundant column:", df.shape)
```

```
Shape after removing redundant column: (3713, 14)
```

As `current_age_y` and `current_age_x` represented the same values but we found out that only `current_age_y` has empty values, then we should remove it. This changes the number of columns in the dataset, so now we have 14 instead of 15 columns.

In [22]:
```
# Check missing values after basic cleaning

print(df.isna().sum())
print("Final shape:", df.shape)
```

```
player_name             0
date_of_birth_age       0
height                 17
position                0
current_age_x           0
minutes_played          9
season                  9
injury               1937
from_date            1937
to_date              1939
days_out             1937
matches_missed       2037
injury_year          1937
age_at_injury        1937
dtype: int64
Final shape: (3713, 14)
```

Final Data Cleaning Check

After the basic cleaning steps, the dataset contains 3,713 rows and 14 columns. Empty strings were standardised as missing values, special values in `matches_missed` (`-` and `?`) were converted to missing values, and the main numerical and date variables were converted to appropriate data types. The `current_age_y` column was also removed.

The final missing-value check shows that some missing values remain in the dataset. These values are not filled at this stage because their meaning requires further investigation and interpretation. In particular, missing injury-related values are not automatically treated as indicating that a player had no injury.

The dataset is now sufficiently prepared for the next stages of the project.

In [23]:
```
# Save the cleaned dataset
# Save the cleaned dataset
import os

os.makedirs("../data/processed", exist_ok=True)

df.to_csv("../data/processed/laliga_cleaned.csv", index=False)
```